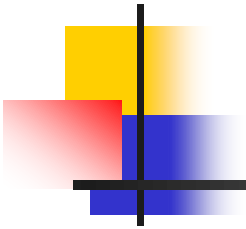


Deterministic Finite Automata (DFA)

Reading: Chapter 2



Finite Automaton (FA)

- Informally, a state diagram that comprehensively captures all possible states and transitions that a machine can take while responding to a stream or sequence of input symbols
- Recognizer for “Regular Languages”

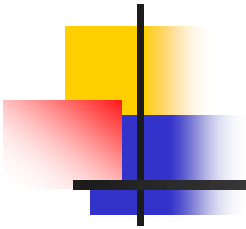
- Deterministic Finite Automata (DFA)
  - The machine can exist in only one state at any given time
- Non-deterministic Finite Automata (NFA)
  - The machine can exist in multiple states at the same time

Deterministic Finite Automata - Definition

- A Deterministic Finite Automaton (DFA) consists of:
  - Q ==> a finite set of states
  - $\sum$ ==> a finite set of input symbols (alphabet)
  - q_0 ==> a start state
  - F ==> set of accepting states
  - δ ==> a transition function, which is a mapping between Q x $\sum$ ==> Q
- A DFA is defined by the 5-tuple:
  - {Q, $\sum$, q_0,F, δ }

What does a DFA do on reading an input string?

- Input: a word w in $\sum^*$
- Question: Is w acceptable by the DFA?
- Steps:
  - Start at the "start state" q_0
  - For every input symbol in the sequence w do
    - Compute the next state from the current state, given the current input symbol in w and the transition function
  - If after all symbols in w are consumed, the current state is one of the accepting states (F) then *accept w;*
  - Otherwise, *reject w.*

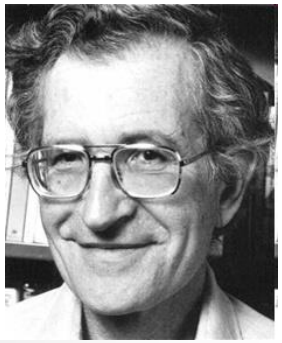


Regular Languages

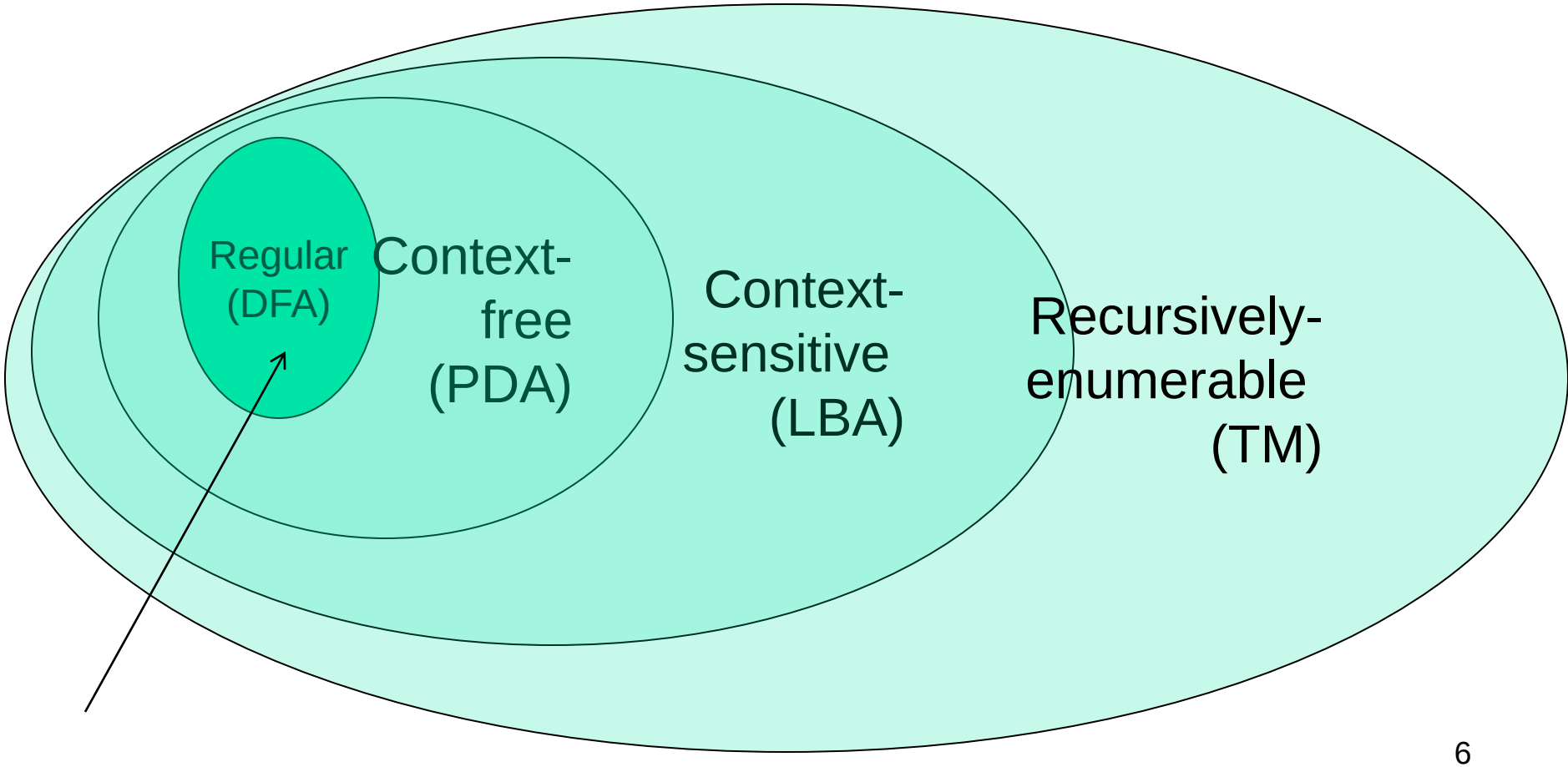
- Let L(A) be a language *recognized* by a DFA A.
  - Then L(A) is called a "*Regular Language*".

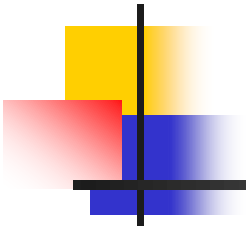
- Locate regular languages in the Chomsky Hierarchy

The Chomsky Hierachy



- A containment hierarchy of classes of formal languages





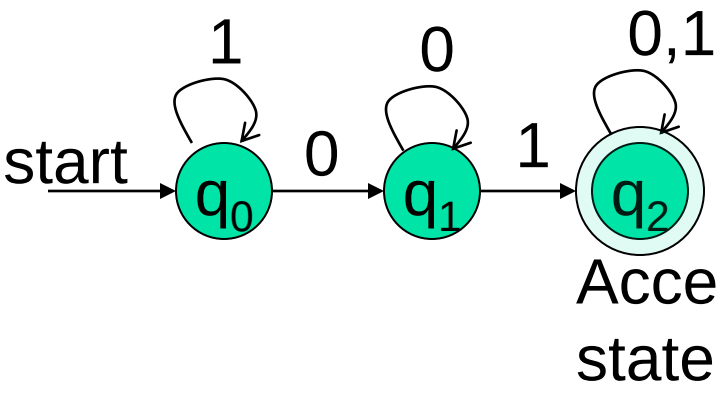
Example #1

- Build a DFA for the following language:
  - L = {w | w is a binary string that contains 01 as a substring}
- Steps for building a DFA to recognize L:
  - ∑ = {0,1}
  - Decide on the states: Q
  - Designate start state and final state(s)
  - δ: Decide on the transitions:
- “Final” states == same as “accepting states”
- Other states == same as “non-accepting states”

Regular expression: (0+1)*01(0+1)*

DFA for strings containing 01

- What makes this DFA deterministic?

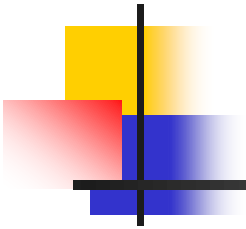


- What if the language allows empty strings?

- $Q = \{q_0, q_1, q_2\}$
- $\Sigma = \{0,1\}$
- start state = q_0
- $F = \{q_2\}$
- Transition table

symbols

states	δ	0	1
→	q_0	q_1	q_0
	q_1	q_1	q_2
	$*q_2$	q_2	q_2



Example #2

Clamping Logic:

- A clamping circuit waits for a ”1” input, and turns on forever. However, to avoid clamping on spurious noise, we’ll design a DFA that waits for *two consecutive 1s* in a row before clamping on.

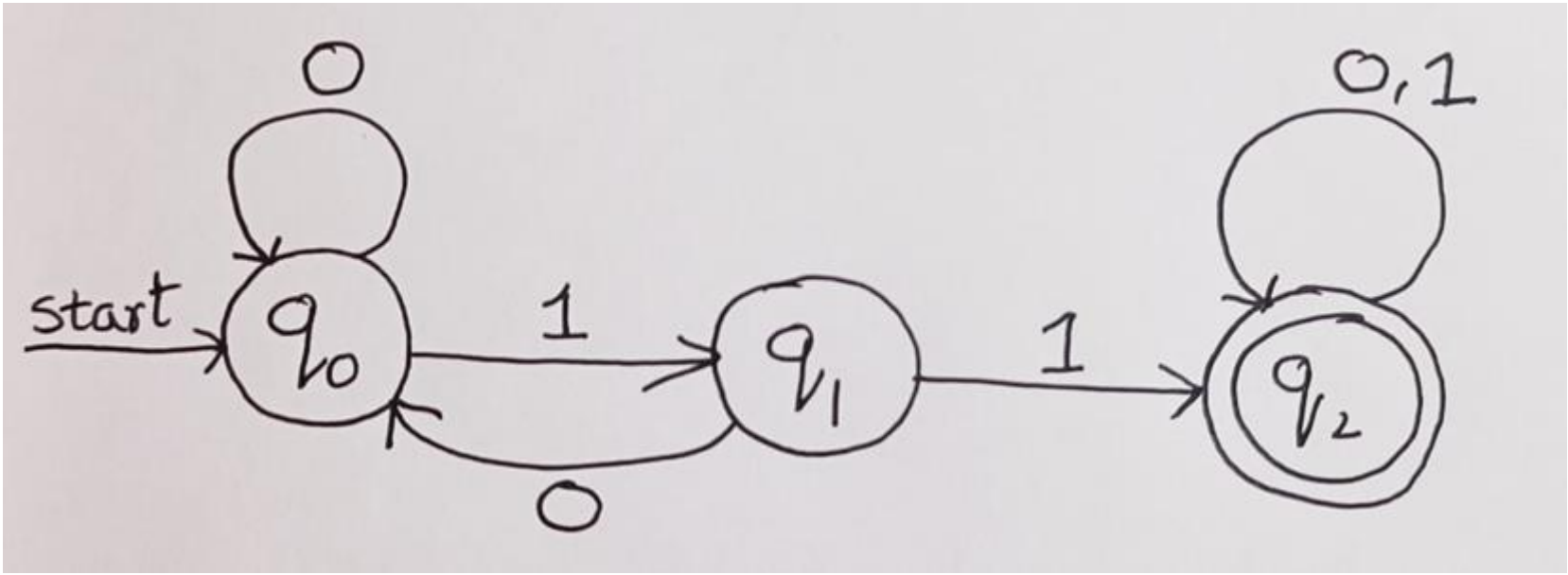
- Build a DFA for the following language:

  L = { w | w is a bit string which contains the substring 11}

- State Design:
  - q_0 : start state (initially off), also means the most recent input was not a 1
  - q_1: has never seen 11 but the most recent input was a 1
  - q_2: has seen 11 at least once

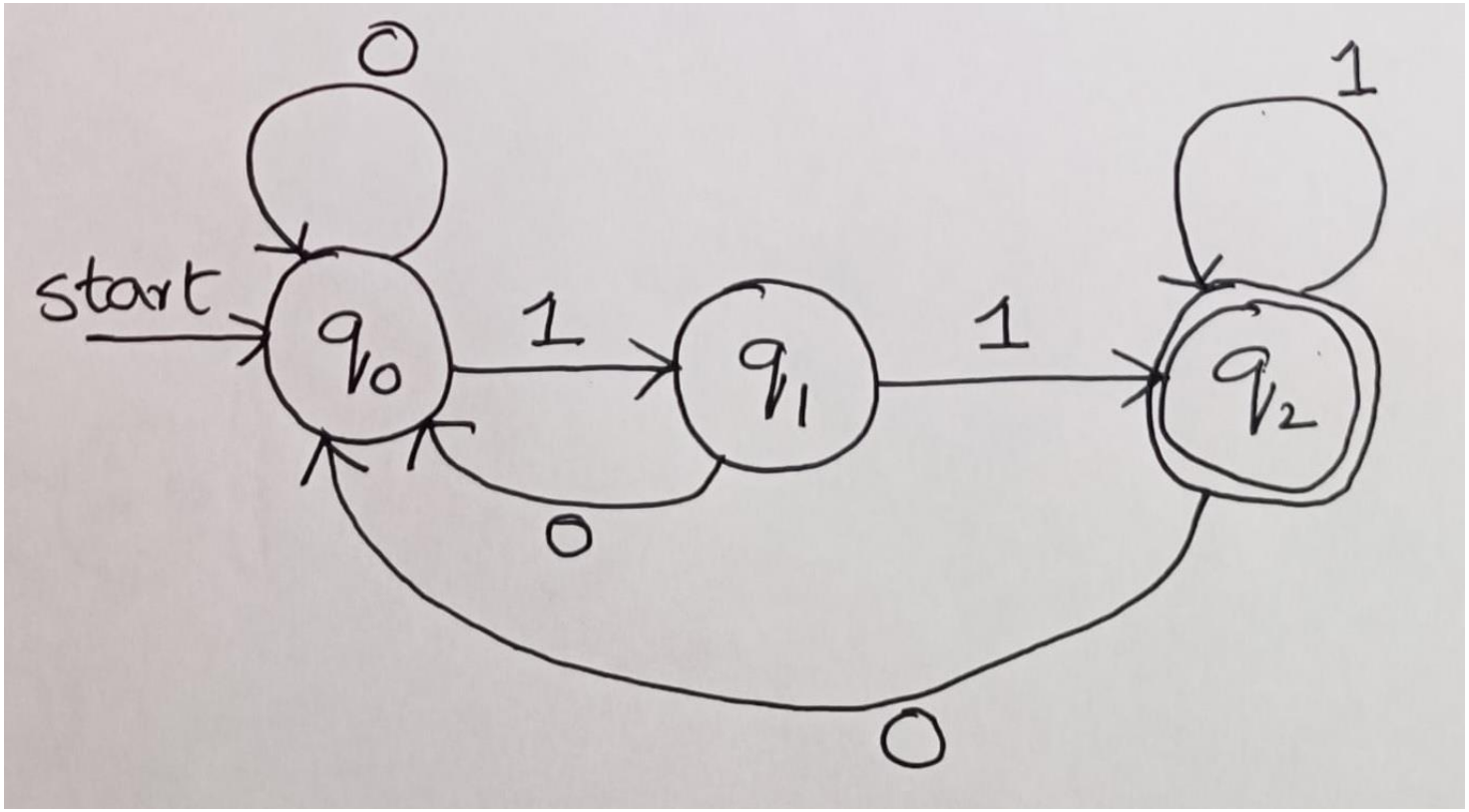
Example #2

- L = { w | w is a bit string which contains the substring 11}



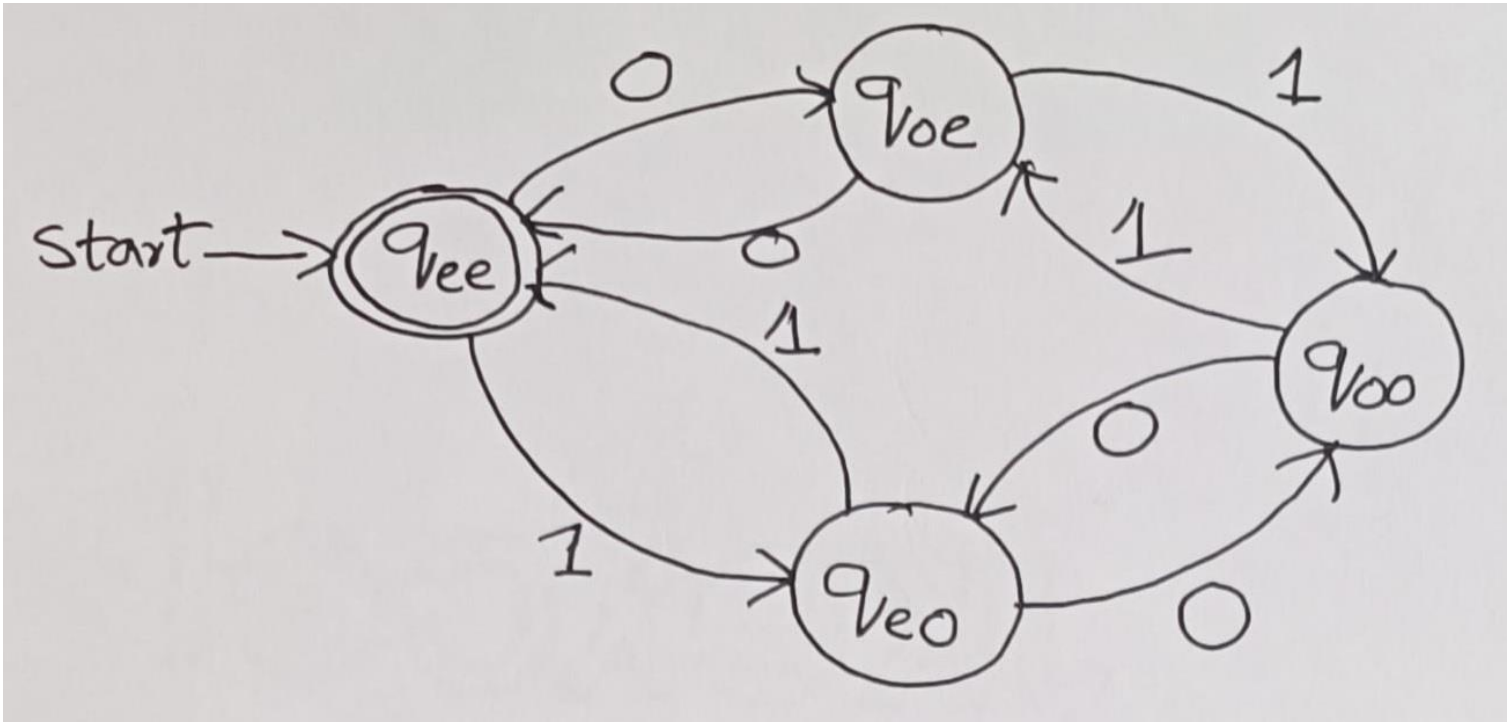
Example #3

- L = { w | w is a bit string which ends with 11}



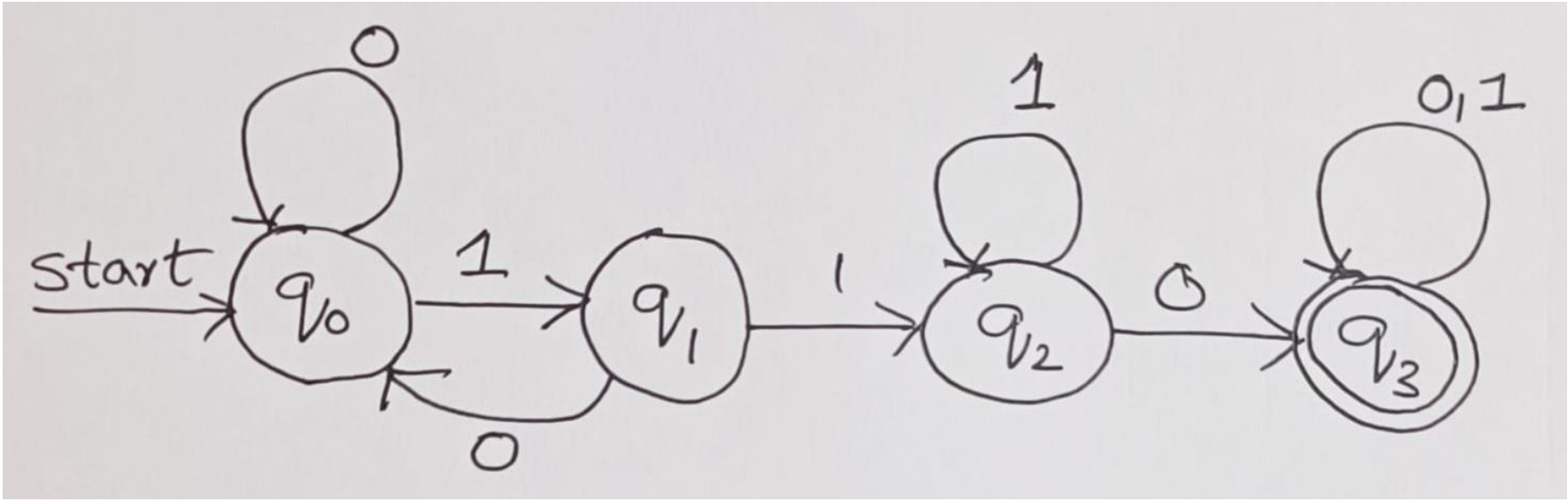
Example #4

- L = { w | w is a binary string that has even number of 1s and even number of 0s}



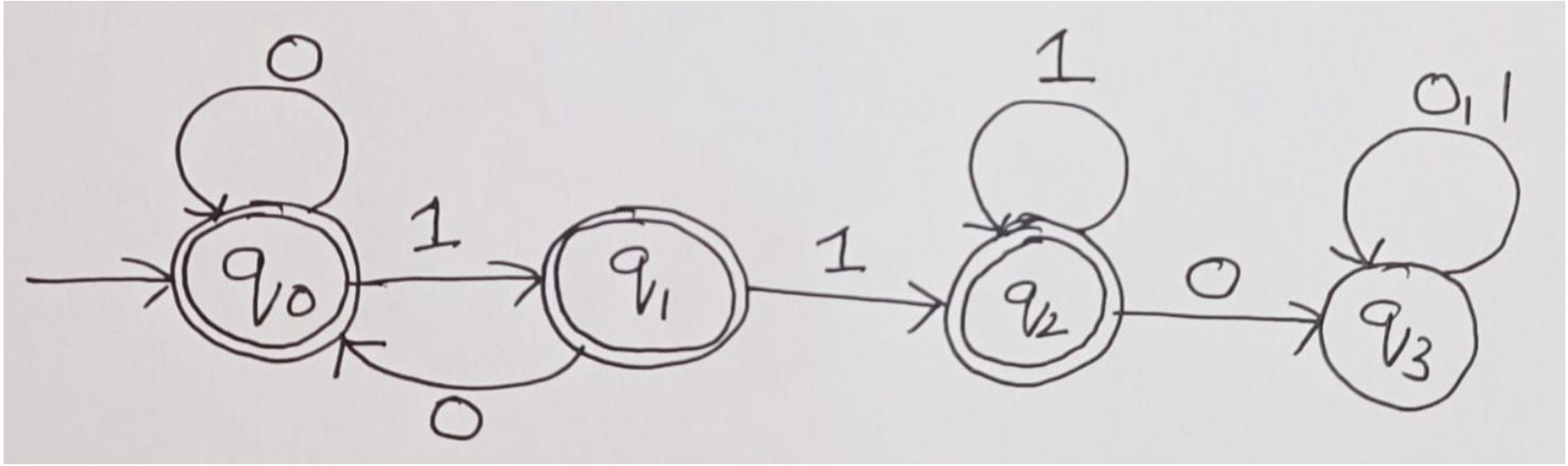
Example #5

- L = { w | w is a binary string that has 110 as substring}



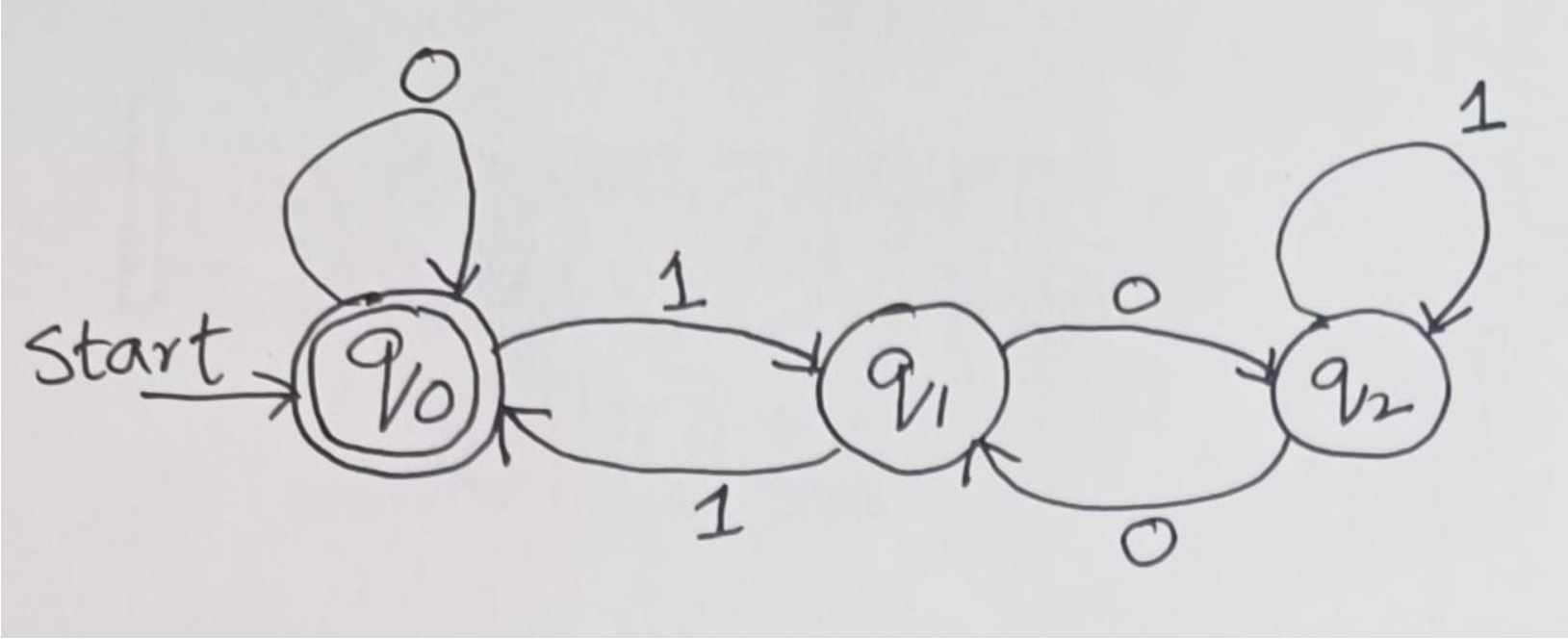
Example #6

- L = { w | w is a binary string that does not contain 110 as substring}



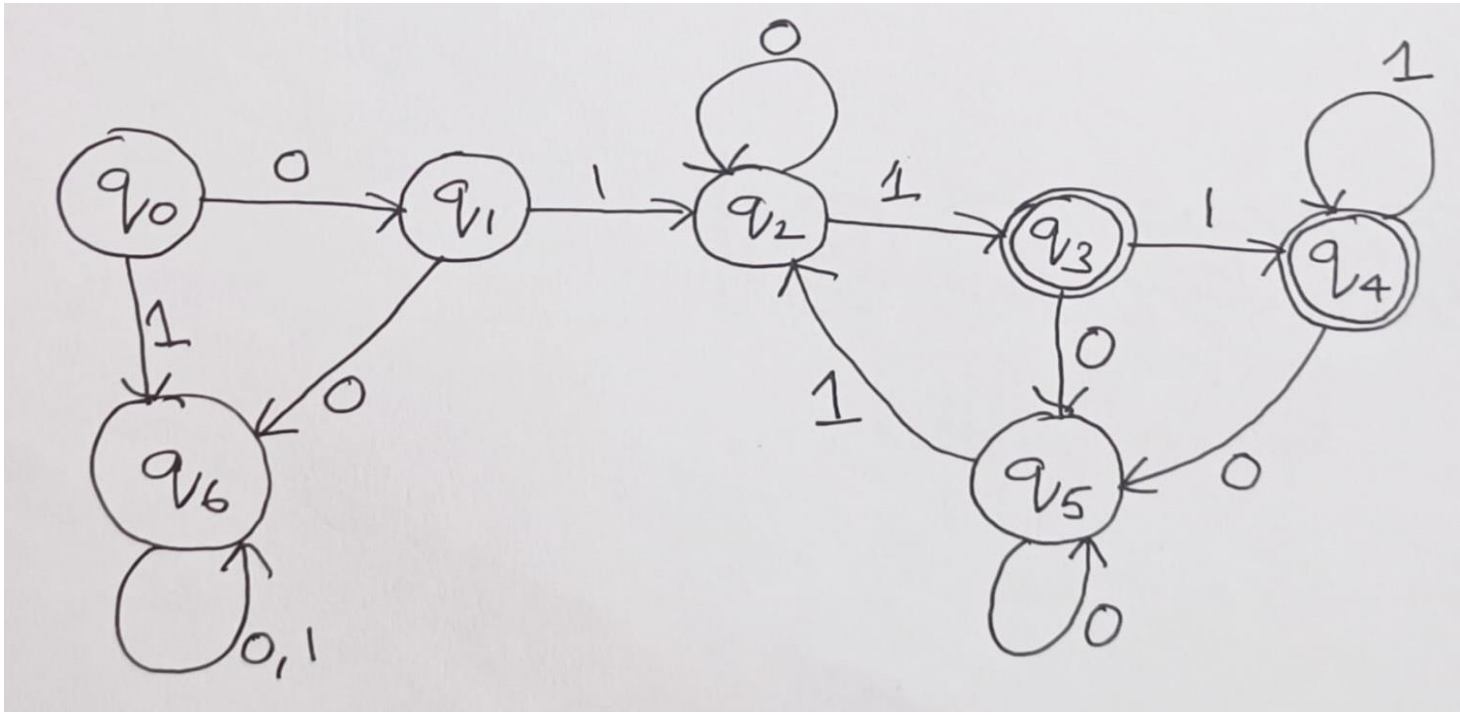
Example #7

- L = { w | w is a binary string whose decimal equivalent is divisible by 3}



Example #8

- L = { w | w is a binary string which starts with 10 and ends with 11}



Extension of transitions (δ) to Paths ($\hat{\delta}$)

- $\hat{\delta}$ *(q,w)* = *destination state* from state *q* on input string *w*

- $\hat{\delta}$ *(q,wa)* = δ ($\hat{\delta}$*(q,w), a)*

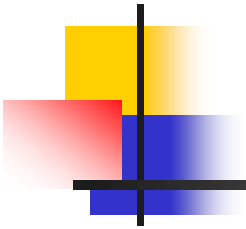
  - Work out example #3 using the input sequence w=10010, a=1:

    - $\hat{\delta}$ (q_0,wa) = *?*

Language of a DFA

A DFA A accepts string *w* if there is a path from q_0 to an accepting (or final) state that is labeled by *w*

- *i.e.,* $L(A) = \{ w \mid \hat{\delta}(q_0,w) \in F \}$
- *I.e., L(A) = all strings that lead to an accepting state from* q_0



Further Exercises

- Construct DFA for the following:

1. The set of all strings beginning with 101
2. The set of all strings containing 1101 as substring
3. The set of all strings with exactly three consecutive zeros